```
1 !pip install pycountry fbprophet
```

```
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-pa
Collecting crashtest<0.4.0,>=0.3.0 (from clikit<0.7,>=0.6->pystan>=2.14->fbprophet)
  Downloading crashtest-0.3.1-py3-none-any.whl.metadata (748 bytes)
Collecting pastel<0.3.0,>=0.2.0 (from clikit<0.7,>=0.6->pystan>=2.14->fbprophet)
  Downloading pastel-0.2.1-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting pylev<2.0,>=1.3 (from clikit<0.7,>=0.6->pystan>=2.14->fbprophet)
  Downloading pylev-1.4.0-py2.py3-none-any.whl.metadata (2.3 kB)
Collecting appdirs<2.0,>=1.4 (from httpstan<4.14,>=4.13->pystan>=2.14->fbprophet)
  Downloading appdirs-1.4.4-py2.py3-none-any.whl.metadata (9.0 kB)
Collecting marshmallow<4.0,>=3.10 (from httpstan<4.14,>=4.13->pystan>=2.14->fbprophet
  Downloading marshmallow-3.26.2-py3-none-any.whl.metadata (7.3 kB)
Collecting webargs<9.0,>=8.0 (from httpstan<4.14,>=4.13->pystan>=2.14->fbprophet)
  Downloading webargs-8.7.1-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: typing-extensions>=4.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: idna>=2.0 in /usr/local/lib/python3.12/dist-packages (
Downloading pycountry-26.2.16-py3-none-any.whl (8.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.0/8.0 MB 109.5 MB/s eta 0:00:00
Downloading cmdstanpy-0.9.5-py3-none-any.whl (37 kB)
Downloading convertdate-2.4.1-py3-none-any.whl (48 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 48.4/48.4 kB 4.1 MB/s eta 0:00:00
Downloading LunarCalendar-0.0.9-py2.py3-none-any.whl (18 kB)
Downloading pystan-3.10.1-py3-none-any.whl (13 kB)
Downloading setuptools_git-1.2-py2.py3-none-any.whl (10 kB)
Downloading clikit-0.6.2-py2.py3-none-any.whl (91 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 91.8/91.8 kB 7.9 MB/s eta 0:00:00
Downloading ephem-4.2.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manyli
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 72.1 MB/s eta 0:00:00
Downloading httpstan-4.13.0-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.wh
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.6/45.6 MB 19.5 MB/s eta 0:00:00
Downloading pysimdjson-7.0.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.w
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 101.3 MB/s eta 0:00:00
Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Downloading crashtest-0.3.1-py3-none-any.whl (7.0 kB)
Downloading marshmallow-3.26.2-py3-none-any.whl (50 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51.0/51.0 kB 4.0 MB/s eta 0:00:00
Downloading pastel-0.2.1-py2.py3-none-any.whl (6.0 kB)
Downloading pylev-1.4.0-py2.py3-none-any.whl (6.1 kB)
Downloading webargs-8.7.1-py3-none-any.whl (32 kB)
Building wheels for collected packages: fbprophet, pymeeus
  error: subprocess-exited-with-error

  × python setup.py bdist_wheel did not run successfully.
  │ exit code: 1
  ╰─> See above for output.

  note: This error originates from a subprocess, and is likely not a problem with pip
  Building wheel for fbprophet (setup.py) ... error
  ERROR: Failed building wheel for fbprophet
  Running setup.py clean for fbprophet
  Building wheel for pymeeus (setup.py) ... done
  Created wheel for pymeeus: filename=PyMeeus-0.5.12-py3-none-any.whl size=732000 sha
  Stored in directory: /root/.cache/pip/wheels/92/74/5d/5cf1193a619dc315b5b0a15887690
Successfully built pymeeus
Failed to build fbprophet
ERROR: ERROR: Failed to build installable wheels for some pyproject.toml based projec
```

What is Corona Virus(COVID-19)?

Coronavirus is a family of viruses that can cause illness, which can vary from common cold and cough to sometimes more severe disease. SARS-CoV-2 (n-coronavirus) is the new virus of the coronavirus family, which first discovered in 2019, which has not been identified in humans before. It is a contiguous virus which started from Wuhan in December 2019. Which later declared as Pandemic by WHO due to high rate spreads throughout the world. Currently (on date 27 March 2020), this leads to a total of 24K+ Deaths across the globe, including 16K+ deaths alone in Europe.Pandemic is spreading all over the world; it becomes more important to understand about this spread. This NoteBook is an effort to analyze the cumulative data of confirmed, deaths, and recovered cases over time. In this notebook, the main focus is to analyze the spread trend of this virus all over the india.

History of COVID-19 in India

On January 30, India reported its first case of COVID-19 in Kerala, which rose to three cases by February 3; all were students who had returned from Wuhan, China.No significant rise in cases was seen in the rest of February.

On 22 March 2020, India observed a 14-hour voluntary public curfew at the instance of the prime minister Narendra Modi.The government followed it up with lockdowns in 75 districts where COVID cases had occurred as well as all major cities.Further, on 24 March, the prime minister ordered a nationwide lockdown for 21 days, affecting the entire 1.3 billion population of India.

The transmission escalated during March, after several cases were reported all over the country, most of which were linked to people with a travel history to affected countries. On 12 March, a 76-year-old man who had returned from Saudi Arabia became the first victim of the virus in the country.On 4 March, 22 new cases came to light, including those of an Italian tourist group with 14 infected members.But number of cases start increasing dramtically after 19th March, but in the month of April it has been its peak.

Experts suggest the number of infections could be much higher as India's testing rates are among the lowest in the world.The infection rate of COVID-19 in India is reported to be 1.7, significantly lower than in the worst affected countries.

Source: Wikipedia

Library

```
issimport numpy as np
import pandas as pd
from IPython.display import Markdown
from datetime import timedelta
```

```
 5 import json, requests
 6 from datetime import datetime
 7 import glob
 8 import requests
 9 from bs4 import BeautifulSoup
10 import matplotlib.pyplot as plt
11 import plotly.express as px
12 import plotly.graph_objs as go
13 import altair as alt
14 %matplotlib inline
15 import seaborn as sns
16 sns.set()
17 import pycountry
18 from plotly.offline import init_notebook_mode, iplot
19 import plotly.offline as py
20 import plotly.express as ex
21 from plotly.offline import download_plotlyjs,init_notebook_mode,plot,iplot
22 py.init_notebook_mode(connected=True)
23 import folium
24 from folium import plugins
25 plt.style.use("seaborn-talk")
26 plt.rcParams['figure.figsize'] = 8, 5
27 plt.rcParams['image.cmap'] = 'viridis'
28 from fbprophet import Prophet
29 pd.set_option('display.max_rows', None)
30 from math import sin, cos, sqrt, atan2, radians
31 import warnings
32 warnings.filterwarnings("ignore")
33 import os
34 for dirname, _, filenames in os.walk('/kaggle/input'):
35     for filename in filenames:
36         print(os.path.join(dirname, filename))
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_414/3395511044.py in <cell line: 0>()
     15 import seaborn as sns
     16 sns.set()
---> 17 import pycountry
     18 from plotly.offline import init_notebook_mode, iplot
     19 import plotly.offline as py

ModuleNotFoundError: No module named 'pycountry'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

OPEN EXAMPLES

Next steps: Explain error

Uploading Data

```
'''
def get_distance_between_lats_lons(lat1,lon1,lat2,lon2):
# approximate radius of earth in km
        R = 6373
        lat1 = radians(lat1)
        lon1 = radians(lon1)
        lat2 = radians(lat2)
        lon2 = radians(lon2)
        dlon = lon2 - lon1
        dlat = lat2 - lat1
        a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
        c = 2 * atan2(sqrt(a), sqrt(1 - a))
        distance = R * c
        return(distance)
city_wise_coordinates= pd.read_csv('../input/indian-postal-codes/IndiaPostalCodes
city_wise_coordinates['City'] = city_wise_coordinates['City'].str.upper()
city_wise_coordinates['District'] = city_wise_coordinates['District'].str.upper()
city_wise_coordinates['State'] = city_wise_coordinates['State'].str.upper()
district_wise_pin_states= city_wise_coordinates.groupby('District')['PIN','State']
district_wise_lat_lng= city_wise_coordinates.groupby('District')['Lat','Lng'].agg(
district_wise_data_geonames= district_wise_pin_states.merge(district_wise_lat_lng,
dfp= pd.read_json("https://api.covid19india.org/raw_data.json")## data from covid1
df3= []
for row in range(0,dfp.shape[0]):
    df1= dfp['raw_data'][row]
    df2=pd.DataFrame(df1.items()).set_index(0)
    df3.append(df2.T)
appended_data = pd.concat(df3, sort=False)
appended_data.replace(r'^\s*$', np.nan, regex=True, inplace = True)
appended_data.rename(columns={'detectedcity':'City'}, inplace=True)
appended_data.rename(columns={'detecteddistrict':'District'}, inplace=True)
appended_data.rename(columns={'detectedstate':'State'}, inplace=True)
appended_data['City'] = appended_data['City'].str.upper()
appended_data['District'] = appended_data['District'].str.upper()
appended_data['State'] = appended_data['State'].str.upper()
appended_data= appended_data.dropna(thresh=3)
district_wise_counts= appended_data.groupby('District').agg({'patientnumber': 'cou
district_wise_counts.rename(columns={'patientnumber':'d_patient_counts'}, inplace=
district_wise_counts =district_wise_counts.reset_index()
district_wise_counts['District'] = district_wise_counts['District'].str.upper()
corona_db_with_latlng= district_wise_counts.merge(district_wise_data_geonames, lef
corona_db_with_latlng.rename(columns={'d_patient_counts':'Num_Positive_cases'}, in
def get_idx_distance_from_query_locations(q_lat, q_lng, corona_db_with_latlng):
    dist_array=[]
    for index, row in corona_db_with_latlng.iterrows():
        dist= int(get_distance_between_lats_lons(q_lat,q_lng,row['Lat'],row['Lng']
        dist_array.append(dist)
    minpos = dist_array.index(min(dist_array))
    mindist= dist_array[minpos]
    cases= corona_db_with_latlng.loc[minpos,'Num_Positive_cases']
    location= corona_db_with_latlng.loc[minpos,'District']
    state= corona_db_with_latlng.loc[minpos,'State']
    Lats= corona_db_with_latlng.loc[minpos,'Lat']
    Lngs= corona_db_with_latlng.loc[minpos,'Lng']
```

```
    return(mindist, cases, location, state)
def get_nearest_covid19_stats(query_info,corona_db_with_latlng):
    if query_info.PIN.iloc[1] in corona_db_with_latlng['PIN'].values:
        mindist= 2
        Lat= corona_db_with_latlng.loc[corona_db_with_latlng.PIN==query_info.PIN.i
        Lng= corona_db_with_latlng.loc[corona_db_with_latlng.PIN==query_info.PIN.i
        mindist= int(get_distance_between_lats_lons(query_info.Lat.iloc[1] ,query_
        cases= corona_db_with_latlng.loc[corona_db_with_latlng.PIN==query_info.PIN
        district= corona_db_with_latlng.loc[corona_db_with_latlng.PIN==query_info.
        state= corona_db_with_latlng.loc[corona_db_with_latlng.PIN==query_info.PIN
        print("The nearest location with COVID-19 from your PIN is in your own Pos
        print("Location: {} , {}".format(district.values[0].upper(), state.values[
    else:
        (mindist, cases, district, state) = get_idx_distance_from_query_locations(
        print("The nearest location with COVID-19 from your PIN is within {} km wi
        print("Location: {} , {}".format(district.upper(), state.upper()))

'''
```

```
link = 'https://www.mohfw.gov.in/'
req = requests.get(link)
soup = BeautifulSoup(req.content, "html.parser")
thead = soup.find_all('thead')[-1]
head = thead.find_all('tr')
tbody = soup.find_all('tbody')[-1]
body = tbody.find_all('tr')
head_rows = []
body_rows = []
for tr in head:
    td = tr.find_all(['th', 'td'])
    row = [i.text for i in td]
    head_rows.append(row)
for tr in body:
    td = tr.find_all(['th', 'td'])
    row = [i.text for i in td]
    body_rows.append(row)
df_bs = pd.DataFrame(body_rows[:len(body_rows)-1], columns=head_rows[0])
df_bs.drop('S. No.', axis=1, inplace=True)
#To remove last raw
df_bs.drop(df_bs.tail(5).index,axis = 0,inplace=True)
#df_bs.drop(df_bs.tail(1).index,axis = 0,inplace=True)
#df_bs.drop(df_bs.tail(1).index,axis = 0,inplace=True)
now  = datetime.now()
df_bs['Date'] = now.strftime("%m/%d/%Y")
df_bs['Date'] = pd.to_datetime(df_bs['Date'], format='%m/%d/%Y')
df_bs.rename(columns = {'Deaths**':'Death'},inplace = True)
#df_bs
```

```
locations = {
    "Kerala" : [10.8505,76.2711],
    "Maharashtra" : [19.7515,75.7139],
    "Karnataka": [15.3173,75.7139],
    "Telangana": [18.1124,79.0193],
    "Uttar Pradesh": [26.8467,80.9462],
    "Rajasthan": [27.0238,74.2179],
    "Gujarat":[22.2587,71.1924],
    "Delhi" : [28.7041,77.1025],
```

```
    "Punjab":[31.1471,75.3412],
    "Tamil Nadu": [11.1271,78.6569],
    "Haryana": [29.0588,76.0856],
    "Madhya Pradesh":[22.9734,78.6569],
    "Jammu and Kashmir":[33.7782,76.5762],
    "Ladakh": [34.1526,77.5770],
    "Andhra Pradesh":[15.9129,79.7400],
    "West Bengal": [22.9868,87.8550],
    "Bihar": [25.0961,85.3131],
    "Chhattisgarh":[21.2787,81.8661],
    "Chandigarh":[30.7333,76.7794],
    "Uttarakhand":[30.0668,79.0193],
    "Himachal Pradesh":[31.1048,77.1734],
    "Goa": [15.2993,74.1240],
    "Odisha":[20.9517,85.0985],
    "Andaman and Nicobar Islands": [11.7401,92.6586],
    "Puducherry":[11.9416,79.8083],
    "Manipur":[24.6637,93.9063],
    "Mizoram":[23.1645,92.9376],
    "Assam":[26.2006,92.9376],
    "Meghalaya":[25.4670,91.3662],
    "Tripura":[23.9408,91.9882],
    "Arunachal Pradesh":[28.2180,94.7278],
    "Jharkhand" : [23.6102,85.2799],
    "Nagaland": [26.1584,94.5624],
    "Sikkim": [27.5330,88.5122],
    "Dadra and Nagar Haveli and Daman and Diu":[20.1809,73.0169],
    "Lakshadweep":[10.5667,72.6417],
    "Daman and Diu":[20.4283,72.8397] ,
    'State Unassigned':[0,0]
}
```

```
lat = {'Delhi':28.7041,
       'Haryana':29.0588,
       'Kerala':10.8505,
       'Rajasthan':27.0238,
       'Telengana':18.1124,
       'Uttar Pradesh':26.8467,
       'Ladakh':34.2996,
       'Tamil Nadu':11.1271,
       'Jammu and Kashmir':33.7782,
       'Punjab':31.1471,
       'Karnataka':15.3173,
       'Maharashtra':19.7515,
       'Andhra Pradesh':15.9129,
       'Odisha':20.9517,
       'Uttarakhand':30.0668,
       'West Bengal':22.9868,
       'Puducherry': 11.9416,
       'Chandigarh': 30.7333,
       'Chhattisgarh':21.2787,
       'Gujarat': 22.2587,
       'Himachal Pradesh': 31.1048,
       'Madhya Pradesh': 22.9734,
       'Bihar': 25.0961,
       'Manipur':24.6637,
       'Mizoram':23.1645,
        'Goa':15.2993,
```

```
     'Andaman and Nicobar Islands':11.7401,
      "Jharkhand" : 23.6102,
      'Arunachal Pradesh': 28.2180,
      'Assam' : 26.2006,
      'Tripura':23.9408,
      'Meghalaya':25.4670,
      'Nagaland#':26.1584}

long = {'Delhi':77.1025,
        'Haryana':76.0856,
        'Kerala':76.2711,
        'Rajasthan':74.2179,
        'Telengana':79.0193,
        'Uttar Pradesh':80.9462,
        'Ladakh':78.2932,
        'Tamil Nadu':78.6569,
        'Jammu and Kashmir':76.5762,
        'Punjab':75.3412,
        'Karnataka':75.7139,
        'Maharashtra':75.7139,
        'Andhra Pradesh':79.7400,
        'Odisha':85.0985,
        'Uttarakhand':79.0193,
        'West Bengal':87.8550,
        'Puducherry': 79.8083,
        'Chandigarh': 76.7794,
        'Chhattisgarh':81.8661,
        'Gujarat': 71.1924,
        'Himachal Pradesh': 77.1734,
        'Madhya Pradesh': 78.6569,
        'Bihar': 85.3131,
        'Manipur':93.9063,
        'Mizoram':92.9376,
         'Goa':74.1240,
        "Jharkhand" : 85.2799,
       'Andaman and Nicobar Islands':92.6586,
       'Arunachal Pradesh' :94.7278,
        'Assam' : 92.9376,
        'Tripura':91.9882,
        'Meghalaya':91.3662,
        'Nagaland#':94.5624
       }
df_bs['Latitude'] = df_bs['Name of State / UT'].map(lat)
df_bs['Longitude'] = df_bs['Name of State / UT'].map(long)
df_bs['Total cases'] = df_bs.iloc[:,1]
df_bs.ix[24, 'Death'] = 0
#df_bs['Death'] = df_bs.iloc[:,3]
#data['Active cases'] = data['Total cases'] - (data['Cured/Discharged/Migrated'] +
```

```
# complete data

file_name = now.strftime("%Y_%m_%d")+'.csv'
file_loc = ''
df_bs.to_csv(file_loc + file_name, index=False)
loc = ""
files = glob.glob(loc+'2020*.csv')
dfs = []
for i in files:
```

```
    df_temp = pd.read_csv(i)
    df_temp = df_temp.rename(columns={'Cured':'Cured/Discharged'})
    df_temp = df_temp.rename(columns={'Cured/Discharged':'Cured/Discharged/Migrate
                                     'Deaths ( more than 70% cases due to comorbi
    dfs.append(df_temp)

# print(dfs)

complete_data = pd.concat(dfs, ignore_index=True).sort_values(['Date'], ascending=
complete_data['Date'] = pd.to_datetime(complete_data['Date'])
complete_data = complete_data.sort_values(['Date', 'Name of State / UT']).reset_ir
cols = ['Total cases','Cured/Discharged/Migrated', 'Death']
tot = complete_data.iloc[:,1]
complete_data[cols] = complete_data[cols].fillna(0).astype('int')
```

```
symptoms={'symptom':['Fever',
        'Dry cough',
        'Fatigue',
        'Sputum production',
        'Shortness of breath',
        'Muscle pain',
        'Sore throat',
        'Headache',
        'Chills',
        'Nausea or vomiting',
        'Nasal congestion',
        'Diarrhoea',
        'Haemoptysis',
        'Conjunctival congestion'],'percentage':[87.9,67.7,38.1,33.4,18.6,14.8,13.

symptoms=pd.DataFrame(data=symptoms,index=range(14))
```

```
data = pd.DataFrame(complete_data)
covid_19_India = pd.read_csv('../input/covid19-in-india/covid_19_india.csv')
df = pd.read_csv('../input/covid19-corona-virus-india-dataset/complete.csv', parse
p_df = pd.read_csv('../input/covid19-corona-virus-india-dataset/patients_data.csv'
p_df['date_announced'] = pd.to_datetime(p_df['date_announced'], errors = 'coerce')
p_df['date_announced'] = pd.to_datetime(p_df['date_announced'], format='%d/%m/%Y')
p_df['status_change_date'] = pd.to_datetime(p_df['status_change_date'], format='%d
p_df['nationality'] = p_df['nationality'].replace('Indian', 'India')
india_data_json = requests.get('https://api.rootnet.in/covid19-in/unofficial/covid
df_india = pd.io.json.json_normalize(india_data_json['data']['statewise'])
df_india = df_india.set_index("state")
test1 = pd.read_csv('../input/icmr-testing-data/testing data.csv')
test1.drop(df.index[[42]],inplace = True)
state_data = pd.read_csv('https://api.covid19india.org/csv/latest/state_wise.csv')
zones = pd.read_csv('../input/covid19-corona-virus-india-dataset/zones.csv')
#testing_data = pd.read_csv('../input/covid19-corona-virus-india-dataset/tests.csv
```

Data Cleaning

```
'''
date_wise_testing = testing_data[['state',"updatedon","totaltested","positive","ne
date_wise_testing['updatedon'] = date_wise_testing['updatedon'].apply(pd.to_dateti
```

```
date_wise_testing = date_wise_testing.groupby(["updatedon"]).sum().reset_index()
def formatted_text(string):
    display(Markdown(string))
date_wise_testing.to_csv('date_wise_testing.csv''

'''
```

```
covid_19_India[covid_19_India['Deaths']=='0#']

```

```
covid_19_India.replace(np.NaN, 0, inplace=True)
covid_19_India.replace('0#', 0, inplace=True)
#covid_19_India.drop(['ConfirmedIndianNational','ConfirmedForeignNational'],axis=1
covid_19_India.fillna(0)
covid_19_India['Confirmed'].astype(int)
covid_19_India['Cured'].astype(int)
covid_19_India['Deaths'].astype(int)
covid_19_India['Active']= covid_19_India['Confirmed']- (covid_19_India['Cured'] +

```

```
data['Active cases'] = data['Total cases'] - (data['Cured/Discharged/Migrated'] +
data.to_csv('state_wise_data.csv',index=False)
```

```
df['Name of State / UT'] = df['Name of State / UT'].str.replace('Union Territory c
df = df[['Date', 'Name of State / UT', 'Latitude', 'Longitude', 'Total Confirmed c
df.columns = ['Date', 'State/UT', 'Latitude', 'Longitude', 'Confirmed', 'Deaths',

for i in ['Confirmed', 'Deaths', 'Cured']:
    df[i] = df[i].astype('int')

df['Active'] = df['Confirmed'] - df['Deaths'] - df['Cured']
df['Mortality rate'] = df['Deaths']/df['Confirmed']
df['Recovery rate'] = df['Cured']/df['Confirmed']

df = df[['Date', 'State/UT', 'Latitude', 'Longitude', 'Confirmed', 'Active', 'Deat
```

```
latest = df[df['Date']==max(df['Date'])]

# days
latest_day = max(df['Date'])
day_before = latest_day - timedelta(days = 1)

# state and total cases
latest_day_df = df[df['Date']==latest_day].set_index('State/UT')
day_before_df = df[df['Date']==day_before].set_index('State/UT')
temp = pd.merge(left = latest_day_df, right = day_before_df, on='State/UT', suffix
latest_day_df['New cases'] = temp['Confirmed_lat'] - temp['Confirmed_bfr']
latest = latest_day_df.reset_index()
latest.fillna(1, inplace=True)
latest.to_csv('statewise_data_with_new_cases.csv')
```

```
date_wise_data = covid_19_India[['State/UnionTerritory',"Date","Confirmed","Deaths
date_wise_data['Date'] = date_wise_data['Date'].apply(pd.to_datetime, dayfirst=Tru
date_wise_data = date_wise_data.groupby(["Date"]).sum().reset_index()
def formatted_text(string):
```

```
    display(Markdown(string))
date_wise_data.to_csv('date_wise_data.csv')
```

```
#df_india.drop(df_india.index[14], inplace=True)
df_india["Lat"] = ""
df_india["Long"] = ""
for index in df_india.index :
    df_india.loc[df_india.index == index,"Lat"] = locations[index][0]
    df_india.loc[df_india.index == index,"Long"] = locations[index][1]
```

```
test1['day'] = test1['day'].apply(pd.to_datetime, dayfirst=True)
test1["positive_ratio"]= np.round(100*test1["totalPositiveCases"]/test1["totalSamp
test1["perday_positive"] = test1["totalPositiveCases"].diff()
test1["perday_tests"] = test1["totalSamplesTested"].diff()
test1["positive_ratio"]= np.round(100*test1["perday_positive"]/test1["perday_tests
test1 = test1.fillna(0)
test1.drop(test1.head(1).index,axis = 0,inplace=True)
test1.to_csv('ICMR_Testing_Data.csv')
#test1.head()
```

Overall view of the situation

```
fig = px.bar(symptoms[['symptom', 'percentage']].sort_values('percentage', ascendi
             x="percentage", y="symptom", color='symptom',color_discrete_sequence
              ,title='Symptom of Coronavirus',orientation='h')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
fig.update_traces(textposition='inside')
fig.update_layout(uniformtext_minsize=8, uniformtext_mode='hide')
fig.update_layout(barmode='stack')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',yaxis_title='Symptoms',xaxis_t
fig.update_layout(template = 'plotly_white')
fig.show()
```

```
total_tested = test1["totalSamplesTested"].max()
total_positive = test1["totalPositiveCases"].max()
#total_tested = date_wise_testing['totaltested'].max()
#total_positive = date_wise_testing['positive'].max()
positivecase_ratio = total_positive * 100 / total_tested
pcr = float("{:.2f}".format(positivecase_ratio))
test_million = np.round(1000000*test1['totalSamplesTested'].max()/13000000000,2)
print('Total Number of people tested :', total_tested)
print('Total Number of positive cases :',total_positive)
print('Test Conducted per Million People :',test_million)
print('Positive case per Tests [%]:',pcr)
print('Total Recovered Cases :',data['Cured/Discharged/Migrated'].sum())
print('Total Deaths :',data['Death'].sum())
```

```
#Overall
fig = go.Figure(data=[go.Pie(labels=['Total Samples Tested','Positive Cases from t
                             values= [total_tested,total_positive],hole =.3)])
fig.update_traces(hoverinfo='label+percent', textinfo='value',textfont_size=20,
                  marker=dict(colors=['#263fa3','#cc3c2f'], line=dict(color='#FFFF
```

```
fig.update_layout(title_text='COVID19 Test Results from ICMR in india',plot_bgcolc
fig.show()
```

```
#Overall
ac= data['Active cases'].sum()
rvd = data['Cured/Discharged/Migrated'].sum()
dth = data['Death'].sum()
fig = go.Figure(data=[go.Pie(labels=['Active','Cured','Death'],
                             values= [ac,rvd,dth],hole =.3)])
fig.update_traces(hoverinfo='label+percent', textinfo='value', textfont_size=20,
                  marker=dict(colors=['#263fa3', '#2fcc41','#cc3c2f'], line=dict(c
fig.update_layout(title_text='Current Situation in India according www.mohfw.gov.i
fig.show()
```

```
Total_confirmed = df_india['active'].sum()
Total_recovered =  df_india['recovered'].sum()
Total_death = df_india['deaths'].sum()
data12 = [['active', Total_confirmed], ['recovered', Total_recovered], ['deaths',
df123 = pd.DataFrame(data12, columns = ['State / UT', 'count'])
fig = px.pie(df123,
             values= 'count',labels=['Active Cases','Cured','Death'],
             names="State / UT",
             title="Real Time data from www.covid19india.org",
             template="seaborn")
fig.update_traces(hoverinfo='label+percent', textinfo='value', textfont_size=14,
                  marker=dict(colors=['#263fa3', '#2fcc41','#cc3c2f'], line=dict(c
fig.update_traces(textposition='inside')
#fig.update_layout(uniformtext_minsize=12, uniformtext_mode='hide')
fig.update_traces(rotation=90, pull=0.05, textinfo="percent+label")
fig.show()
```

```
from IPython.core.display import HTML
HTML(''' <iframe title="" aria-label="Interactive line chart" id="datawrapper-char
</script> ''')
```

```
'''
#Overall
ac= df_india['active'].sum()
rvd = df_india['recovered'].sum()
dth = df_india['deaths'].sum()
fig = go.Figure(data=[go.Pie(labels=['Active Cases','Cured','Death'],
                             values= [ac,rvd,dth],hole =.3)])
fig.update_traces(hoverinfo='label+percent', textinfo='value', textfont_size=20,
                  marker=dict(colors=['#263fa3', '#2fcc41','#cc3c2f'], line=dict(c
fig.update_layout(title_text='Current Situation in India according www.covid19indi
fig.show()

'''
```

Gender wise insights of COVID-19 cases in india

```
gender_wise = p_df[['gender','age_bracket','current_status']]
gender_wise = gender_wise.fillna("unknown")
```

```
male = len(gender_wise[gender_wise['gender'] == 'M'])
female = len(gender_wise[gender_wise['gender'] == 'F'])
unknown = len(gender_wise[gender_wise['gender'] == 'unknown'])
fig = go.Figure(data=[go.Pie(labels=['Male','Female','Unknown'],
                             values= [male,female,unknown],hole =.3)])
fig.update_traces(hoverinfo='label+percent', textinfo='value', textfont_size=20,
                  marker=dict(colors=['#263fa3', '#d461bf','#d5dfe3'], line=dict(c
fig.update_layout(title_text='Gender wise Active Cases (Numbers are inaccurate due
plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
gender_wise = p_df[['gender','age_bracket','current_status']]
gender_wise = gender_wise.fillna("unknown")
recvd = gender_wise[gender_wise['current_status'] == 'Recovered']
male = len(recvd[recvd['gender'] == 'M'])
female = len(recvd[recvd['gender'] == 'F'])
unknown = len(recvd[recvd['gender'] == 'unknown'])
fig = go.Figure(data=[go.Pie(labels=['Male','Female','Unknown'],
                             values= [male,female,unknown],hole =.3)])
fig.update_traces(hoverinfo='label+percent', textinfo='value', textfont_size=20,
                  marker=dict(colors=['#263fa3', '#d461bf','#d5dfe3'], line=dict(c
fig.update_layout(title_text='Gender wise Recovered Cases (Numbers are inaccurate
plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
#option
temp = gender_wise.copy()
dd = temp[temp['current_status'] == 'Deceased']
d_f = len(dd[dd['gender'] == 'F'])
d_m = len(dd[dd['gender'] == 'M'])
unk = len(dd[dd['gender']== 'unknown'])
fig = go.Figure(data=[go.Pie(labels=['Male','Female','Unknown'],
                             values= [d_m,d_f,unk],hole =.3)])
fig.update_traces(hoverinfo='label+percent', textinfo='value', textfont_size=20,
                  marker=dict(colors=['#263fa3','#d461bf','#d5dfe3'], line=dict(co
fig.update_layout(title_text='Gender wise Deaths (Numbers are inaccurate due to mi
fig.show()
```

```
temp = p_df[['status_change_date','age_bracket','current_status','gender','detecte
rec = temp[temp['current_status'] == 'Recovered'].drop('current_status',axis =1).s
rec_x = rec['age_bracket'].astype(int)
rec_y = rec['gender']
fig = px.histogram(x=rec_x,color =rec_y,orientation = 'v',
                   title='Age wise Recovered cases in Male and Female (Numbers are
fig.update_layout(barmode='stack')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',yaxis_title='Deaths',xaxis_tit
fig.show()
```

```
temp = p_df.copy()
temp = p_df[['status_change_date','age_bracket','current_status','gender','detecte
dea = temp[temp['current_status'] == 'Deceased'].drop('current_status',axis =1).sc
xaxis = dea['age_bracket'].astype(int)
yaxis = dea['gender']
fig = px.histogram(x=xaxis,color =yaxis,orientation = 'v',
                   title='Age wise Deaths in Male and Female (Numbers are inaccura
fig.update_layout(barmode='stack')
```

```
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',yaxis_title='Deaths',xaxis_tit
fig.show()
```

Trend of COVID-19 cases in India

```
import plotly.express as px
fig = px.bar(test1, x="day", y="perday_tests", barmode='group',height=500,color =
             orientation = 'v',color_discrete_sequence = px.colors.sequential.Plas
fig.update_layout(title_text='Number of COVID-19 test conducted everyday',plot_bgc
fig.update_layout(barmode='stack')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',yaxis_title='Tests',xaxis_titl
fig.show()
```

```
fig = go.Figure(data=[
go.Bar(name='Tested', x=test1['day'], y=test1['perday_tests'],marker_color='#2fcc4
go.Bar(name='Positive', x=test1['day'], y=test1['perday_positive'],marker_color='#
fig.update_layout(barmode='stack',width=500, height=600)
fig.update_traces(textposition='inside')
fig.update_layout(uniformtext_minsize=8, uniformtext_mode='hide')
fig.update_layout(title_text='Number of people tested and positive among them',
                  plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
temp = date_wise_data.copy()
fig = go.Figure(data=[
go.Bar(name='Deaths', x=temp['Date'], y=temp['Deaths'],marker_color='#ff0000'),
go.Bar(name='Recovered Cases', x=temp['Date'], y=temp['Cured'],marker_color='#2bac
go.Bar(name='Confirmed Cases', x=temp['Date'], y=temp['Confirmed'],marker_color='#
fig.update_layout(barmode='stack')
fig.update_traces(textposition='inside')
fig.update_layout(uniformtext_minsize=8, uniformtext_mode='hide')
fig.update_layout(title_text='COVID-19 Cases,Recovery and Deaths in India',
                  plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
perday2 = date_wise_data.groupby(['Date'])['Confirmed','Cured','Deaths','Active'].
perday2['New Daily Confirmed Cases'] = perday2['Confirmed'].sub(perday2['Confirmed
perday2['New Daily Confirmed Cases'].iloc[0] = perday2['Confirmed'].iloc[0]
perday2['New Daily Confirmed Cases'] = perday2['New Daily Confirmed Cases'].astype
perday2['New Daily Cured Cases'] = perday2['Cured'].sub(perday2['Cured'].shift())
perday2['New Daily Cured Cases'].iloc[0] = perday2['Cured'].iloc[0]
perday2['New Daily Cured Cases'] = perday2['New Daily Cured Cases'].astype(int)
perday2['New Daily Deaths Cases'] = perday2['Deaths'].sub(perday2['Deaths'].shift(
perday2['New Daily Deaths Cases'].iloc[0] = perday2['Deaths'].iloc[0]
perday2['New Daily Deaths Cases'] = perday2['New Daily Deaths Cases'].astype(int)
perday2.to_csv('perday_daily_cases.csv')
```

```
# New COVID-19 cases reported daily in India
import plotly.express as px
fig = px.bar(perday2, x="Date", y="New Daily Confirmed Cases", barmode='group',hei
fig.update_layout(title_text='New COVID-19 cases reported daily in India',plot_bgc
fig.show()
```

```
# New COVID-19 cured cases reported daily in India
import plotly.express as px
fig = px.bar(perday2, x="Date", y="New Daily Cured Cases", barmode='group',height=
            color_discrete_sequence = ['#319146'])
fig.update_layout(title_text='New COVID-19 Recovered cases reported daily in India
fig.show()
```

```
import plotly.express as px
fig = px.bar(perday2, x="Date", y="New Daily Deaths Cases", barmode='group',height
             color_discrete_sequence = ['#e31010'])
fig.update_layout(title_text='New COVID-19 Deaths reported daily in India',plot_bg
fig.show()
```

```
temp = date_wise_data.copy()
temp = date_wise_data.groupby('Date')['Confirmed', 'Deaths', 'Cured'].sum().reset_
fig = px.scatter(temp, x="Date", y="Confirmed", color="Confirmed",
                 size='Confirmed', hover_data=['Confirmed'],
                 color_discrete_sequence = ex.colors.cyclical.IceFire)
fig.update_layout(title_text='Trend of Daily Coronavirus Cases in India',
                  plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
fig.show()
```

```
'''
def to_log(x):
    return np.log(x + 1)

'''
```

```
fig = px.line(date_wise_data, x="Date", y="Confirmed",
              title="Confirmed Cases (Logarithmic Scale) Over Time in India",
              log_y=True,template='gridon',width=600, height=600)
fig.show()
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=date_wise_data['Confirmed'],
                     mode='lines+markers',marker_color='blue',name='Confimned Cases
fig.add_trace(go.Scatter(x=date_wise_data['Date'],y=date_wise_data['Active'],
                mode='lines+markers',marker_color='purple',name='Active Cases'))
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=date_wise_data['Cured'],
                mode='lines+markers',marker_color='green',name='Recovered'))
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=date_wise_data['Deaths'],
                mode='lines+markers',marker_color='red',name='Deaths'))
fig.update_layout(title_text = '<b>Spread of the Coronavirus Over Time in India </
fig.show()
```

```
cnf = '#263fa3' # confirmed - blue
act = '#fe9801' # active case - yellow
rec = '#21bf73' # recovered - green
dth = '#de260d' # death - red
tmp = date_wise_data.melt(id_vars="Date",value_vars=['Deaths','Cured' ,'Active','C
                 var_name='Case',value_name='Count')
fig = px.area(tmp, x="Date", y="Count",color='Case',
              title='Trend of Covid-10 in India over time: Area Plot',color_discre
```

```
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=550, height=600)
fig.show()
```

```
df_india_data = df[['Date', 'State/UT','Confirmed','Cured','Deaths']]
spread = df_india_data.groupby(['Date', 'State/UT'])['Confirmed'].sum().reset_inde
fig = px.area(spread, x="Date", y="Confirmed",color='State/UT',title='State Wise S
              color_discrete_sequence = ex.colors.cyclical.Edge)
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=700, height=600)
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=test1['day'], y=test1['positive_ratio'],
                    mode='lines+markers',marker_color='blue'))
fig.update_layout(title_text = 'Trend of Positive case ratio from tested people of
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
fig.show()
```

```
temp = date_wise_data.copy()
temp['Recovery Rate'] = temp['Cured']/temp['Confirmed']*100
fig = go.Figure()
fig.add_trace(go.Scatter(x=temp['Date'], y=temp['Recovery Rate'],
                    mode='lines+markers',marker_color='green'))
fig.update_layout(title_text = 'Trend of Recovery Rate of India')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
fig.show()
```

```
temp = date_wise_data.copy()
temp['Mortality Rate'] = temp['Deaths']/temp['Confirmed']*100
fig = go.Figure()
fig.add_trace(go.Scatter(x=temp['Date'], y=temp['Mortality Rate'],mode='lines+mark
fig.update_layout(title_text = 'Trend of Mortality Rate of India')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
fig.show()
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=perday2['Date'], y=perday2['New Daily Confirmed Cases']
                    mode='lines+markers',marker_color='blue',name='Confimned Cases
fig.add_trace(go.Scatter(x=perday2['Date'],y=perday2['New Daily Cured Cases'],
                mode='lines+markers',marker_color='green',name='Recovered Cases'))
fig.update_layout(title_text = 'Newly Infected vs. Newly Recovered in India',plot_
fig.show()
```

```
outcome = date_wise_data['Cured'] + date_wise_data['Deaths']
r_ = date_wise_data['Cured']/outcome * 100
d_ = date_wise_data['Deaths']/outcome * 100
fig = go.Figure()
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=r_,mode='lines+markers',marke
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=d_,mode='lines+markers',marke
fig.update_layout(title_text = 'Outcome of total closed cases (recovery rate vs de
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
agegroup = pd.read_csv('../input/covid19-in-india/AgeGroupDetails.csv')
fig = go.Figure()
fig.add_trace(go.Scatter(x=agegroup['AgeGroup'],y=agegroup['TotalCases'],line_shap
```

```
fig.update_layout(title="Age wise Confirmed Case Trend in India",yaxis_title="Tota
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600,height=600)
fig.show()
```

Covid-19 State Wise insight

```
HTML(''' <div class="flourish-embed flourish-bar-chart-race" data-src="visualisati
```

```
'''
statewise_test = pd.read_csv('../input/covid19-in-india/StatewiseTestingDetails.cs
statewise_test = statewise_test.fillna(0)
statewise_test = statewise_test.groupby('State')['TotalSamples', 'Negative', 'Posi
statewise_test.to_csv('State_wise_Testing.csv')
temp = statewise_test.copy()
temp = temp.sort_values('TotalSamples', ascending=False)
state_order = temp['State']
fig = px.bar(temp,x="TotalSamples", y="State", color='State',
             title='State Wise Testing', orientation='h', text='TotalSamples',
             height=900,color_discrete_sequence = ex.colors.cyclical.Edge)
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
#fig.update_layout(template = 'plotly_white')
fig.show('

'''
```

```
temp = latest.copy()
temp= latest[latest['New cases'] > 0]
temp = temp.sort_values('New cases', ascending=False)
state_order = temp['State/UT']
fig = px.bar(temp,x="New cases", y="State/UT", color='State/UT',
             title='State Wise New cases in Last 24hrs', orientation='h', text='Ne
             height=600)
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
#fig.update_layout(template = 'plotly_white')
fig.show()
```

```
temp = data.sort_values('Total cases', ascending=True)
fig = go.Figure(data=[
go.Bar(name='Active', y=temp['Name of State / UT'], x=temp['Active cases'],
       orientation='h',marker_color='#0f5dbd'),
    go.Bar(name='Cured', y=temp['Name of State / UT'], x=temp['Cured/Discharged/Mi
       orientation='h',marker_color='#319146'),
go.Bar(name='Death', y=temp['Name of State / UT'], x=temp['Death'],
       orientation='h',marker_color='#e03216')])
fig.update_layout(barmode='stack',width=600, height=800)
#fig.update_traces(textposition='inside')
fig.update_layout(uniformtext_minsize=8, uniformtext_mode='hide')
fig.update_layout(title_text='Active Cases,Cured,Deaths in Different States of Inc
                  plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
#colors
'''
```

```
color_discrete_sequence = px.colors.sequential.Plasma_r,template = 'plotly_white',
ex.colors.cyclical.IceFire, ex.colors.cyclical.Edge'
'''
```

```
temp = data.copy()
temp = data.sort_values('Total cases', ascending=False)
state_order = temp['Name of State / UT']
fig = px.bar(temp,x="Total cases", y="Name of State / UT", color='Name of State /
             title='State Wise Confirmed Cases', orientation='h', text='Total case
             height=900,color_discrete_sequence = ex.colors.cyclical.IceFire)
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
#fig.update_layout(template = 'plotly_white')
fig.show()
```

```
temp = data.copy()
temp = data[data['Cured/Discharged/Migrated']>0].sort_values('Cured/Discharged/Mig
state_order = temp['Name of State / UT']
fig = px.bar(temp,x="Cured/Discharged/Migrated", y="Name of State / UT", color='Na
             title='State wise Cured/Discharged/Migrated cases', orientation='h',
             text='Cured/Discharged/Migrated',
             height=700,color_discrete_sequence = ex.colors.cyclical.Phase)
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
#fig.update_layout(template = 'plotly_white')
fig.show()
```

```
temp = data.copy()
temp['Recovery Rate'] = round((temp['Cured/Discharged/Migrated']/temp['Total cases
temp = temp[temp['Total cases']>100]
temp = temp.sort_values('Recovery Rate', ascending=False)
fig = px.bar(temp.sort_values(by="Recovery Rate", ascending=False)[:30][::-1],
             x = 'Recovery Rate', y = 'Name of State / UT',
             title='Recoveries per 100 Confirmed Cases', text='Recovery Rate', hei
             color_discrete_sequence=['#2ca02c'])
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

```
temp = data.copy()
temp = data[data['Death']>0].sort_values('Death',ascending=False)
state_order = temp['Name of State / UT']
fig = px.bar(temp,x="Death", y="Name of State / UT", color='Name of State / UT',
             title='State wise Deaths', orientation='h',
             text='Death',
             height=600,color_discrete_sequence = px.colors.sequential.Plasma_r)
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
#fig.update_layout(template = 'plotly_white')
fig.show()
```

```
temp = data.copy()
temp['Mortality Rate'] = round((temp['Death']/temp['Total cases'])*100, 2)
temp = temp[temp['Total cases']>100]
temp = temp.sort_values('Mortality Rate', ascending=False)
fig = px.bar(temp.sort_values(by="Mortality Rate", ascending=False)[:30][::-1],
             x = 'Mortality Rate', y = 'Name of State / UT',
             title='Mortality Rate per 100 Confirmed Cases', text='Mortality Rate'
             color_discrete_sequence=['darkred'])
```

```
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
fig.show()
```

Demographics Visualisation

```
# tiles='cartodbpositron'

india = folium.Map(location=[20.5937, 78.9629], zoom_start=14,max_zoom=4,min_zoom=
                   tiles = "CartoDB dark_matter",detect_retina = True,height = 600
for i in range(0,len(df_india[df_india['confirmed']>0].index)):
    folium.Circle(
        location=[df_india.iloc[i]['Lat'], df_india.iloc[i]['Long']],
        tooltip = "<h5 style='text-align:center;font-weight: bold'>"+df_india.iloc
                    "<hr style='margin:10px;'>"+
                    "<ul style='color: #444;list-style-type:circle;align-item:left
        "<li>Confirmed: "+str(df_india.iloc[i]['confirmed'])+"</li>"+
        "<li>Active:   "+str(df_india.iloc[i]['active'])+"</li>"+
        "<li>Recovered:   "+str(df_india.iloc[i]['recovered'])+"</li>"+
        "<li>Deaths:   "+str(df_india.iloc[i]['deaths'])+"</li>"+
        "<li>Mortality Rate:   "+str(np.round(df_india.iloc[i]['deaths']/(df_india
        "</ul>",
        radius=(int(np.log2(df_india.iloc[i]['confirmed']+1)))*9000,
        color='red',
        fill_color='green',
        fill=True).add_to(india)
india

```

Analyse Affected area through Heatmap

```
from folium.plugins import HeatMap, HeatMapWithTime
affected_area = folium.Map(location=[20.5937, 78.9629], zoom_start=14,max_zoom=4,m
                           tiles='cartodbpositron',height = 500,width = '70%')
HeatMap(data=df_india[['Lat','Long','confirmed']].groupby(['Lat','Long']).sum().re
        radius=18, max_zoom=14).add_to(affected_area)
affected_area
```

COVID-19 Cases thtough time in different parts of india

```
from plotly.offline import init_notebook_mode
import plotly.graph_objs as go
init_notebook_mode(connected=True)
tmp = df.copy()
tmp['Date'] = tmp['Date'].dt.strftime('%Y/%m/%d')
fig = px.scatter_geo(tmp,lat="Latitude", lon="Longitude", color='Confirmed', size=
                     projection="natural earth",
                     hover_name="State/UT", scope='asia', animation_frame="Date",
                     color_continuous_scale=px.colors.diverging.curl,center={'lat'
                     range_color=[0, max(tmp['Confirmed'])])
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)')
fig.show()
tmp.to_csv('covid_19_india.csv')
```

```
HTML ('''<iframe title="COVID-19 confirmed cases by districts of India" aria-label
</script> ''')
```

```
HTML ('''  <iframe title="COVID-19 District Wise Zones" aria-label="Map" id="datawr
</script> ''')
```

```
HTML('''  <iframe title="[Covid-19: India is supplying HCQ, Paracetamol to 108 cour
</script> ''')
```

Enter you PINCODE and Find the nearest corona positive location and the number of cases with risk factor (low, moderate, high)

```
''''
print('Enter your Indian Pincode number to Find the nearest corona positive locati
query_pincode = input("Enter you PINCODE to know status  : ")
g = int(query_pincode)

if g in city_wise_coordinates.PIN.values:
    query_info= city_wise_coordinates[city_wise_coordinates.PIN == int(g)]
    get_nearest_covid19_stats(query_info,corona_db_with_latlng)
else:
    print('You entered an Invalid PIN')

'''

```

Fit a logistic curve

Let's try to fit a Logistic curve for predicting future behavior of the cumulative number of confirmed cases.

- L (the maximum number of confirmed cases) = 250000 taken from the US example (this is from long time obsolete now)
- k (growth rate) = 0.25 approximated value from most of the countries
- x0 (the day of the inflexion) = 80 approximated The curve being:

$$y = \frac{L}{1 + e^{-k(x-x_0)}} + 1$$

```
import scipy
def logistic(x, L, k, x0):
    return L / (1 + np.exp(-k * (x - x0))) + 1
d_df = date_wise_data.copy()
p0 = (0,0,0)
def plot_logistic_fit_data(d_df, title, p0=p0):
    d_df = d_df.sort_values(by=['Date'], ascending=True)
```

```
    d_df['x'] = np.arange(len(d_df)) + 1
    d_df['y'] = d_df['Confirmed']

    x = d_df['x']
    y = d_df['y']

    c2 = scipy.optimize.curve_fit(logistic,  x,  y,  p0=p0 )
    #y = logistic(x, L, k, x0)
    popt, pcov = c2

    x = range(1,d_df.shape[0] + int(popt[2]))
    y_fit = logistic(x, *popt)

    p_df = pd.DataFrame()
    p_df['x'] = x
    p_df['y'] = y_fit.astype(int)

    print("Predicted L (the maximum number of confirmed cases): " + str(int(popt[0
    print("Predicted k (growth rate): " + str(float(popt[1])))
    print("Predicted x0 (the day of the inflexion): " + str(int(popt[2])) + "")

    x0 = int(popt[2])

    traceC = go.Scatter(
        x=d_df['x'], y=d_df['y'],
        name="Confirmed",
        marker=dict(color="Red"),
        mode = "markers+lines",
        text=d_df['Confirmed'],
    )

    traceP = go.Scatter(
        x=p_df['x'], y=p_df['y'],
        name="Predicted",
        marker=dict(color="blue"),
        mode = "lines",
        text=p_df['y'],
    )

    trace_x0 = go.Scatter(
        x = [x0, x0], y = [0, p_df.loc[p_df['x']==x0,'y'].values[0]],
        name = "X0 - Inflexion point",
        marker=dict(color="black"),
        mode = "lines",
        text = "X0 - Inflexion point"
    )

    data = [traceC, traceP, trace_x0]

    layout = dict(title = 'Cumulative Conformed cases and logistic curve projectio
          xaxis = dict(title = 'Day since first case', showticklabels=True),
          yaxis = dict(title = 'Number of cases'),
          hovermode = 'closest',plot_bgcolor='rgb(275, 270, 273)'
         )
    fig = dict(data=data, layout=layout)
    iplot(fig, filename='covid-logistic-forecast')

L = 250000
k = 0.25
```

```
x0 = 100
p0 = (L, k, x0)
plot_logistic_fit_data(d_df, 'India')
```

Fitting an exponential curve

The parameters for the curve are:

- A - the constant multiplier for the exponential
- B - the multiplier for the exponent

The curve is thus:

$$y = Ae^{Bx}$$

```
import datetime
import scipy
p0 = (0,0)
def plot_exponential_fit_data(d_df, title, delta, p0):
    d_df = d_df.sort_values(by=['Date'], ascending=True)
    d_df['x'] = np.arange(len(d_df)) + 1
    d_df['y'] = d_df['Confirmed']

    x = d_df['x'][:-delta]
    y = d_df['y'][:-delta]

    c2 = scipy.optimize.curve_fit(lambda t,a,b: a*np.exp(b*t),  x,  y,  p0=p0)

    A, B = c2[0]
    print(f'(y = Ae^(Bx)) A: {A}, B: {B}')
    x = range(1,d_df.shape[0] + 1)
    y_fit = A * np.exp(B * x)

    traceC = go.Scatter(
        x=d_df['x'][:-delta], y=d_df['y'][:-delta],
        name="Confirmed (included for fit)",
        marker=dict(color="Red"),
        mode = "markers+lines",
        text=d_df['Confirmed'],
    )

    traceV = go.Scatter(
        x=d_df['x'][-delta-1:], y=d_df['y'][-delta-1:],
        name="Confirmed (validation)",
        marker=dict(color="blue"),
        mode = "markers+lines",
        text=d_df['Confirmed'],
    )

    traceP = go.Scatter(
        x=np.array(x), y=y_fit,
        name="Projected values (fit curve)",
        marker=dict(color="green"),
        mode = "lines",
        text=y_fit,
    )

```

```
    data = [traceC, traceV, traceP]

    layout = dict(title = 'Cumulative Conformed cases and exponential curve projec
          xaxis = dict(title = 'Day since first case', showticklabels=True),
          yaxis = dict(title = 'Number of cases'),plot_bgcolor='rgb(275, 270, 273)
          hovermode = 'closest'
         )
    fig = dict(data=data, layout=layout)
    iplot(fig, filename='covid-exponential-forecast')
p0 = (40, 0.2)
plot_exponential_fit_data(d_df, 'I', 7, p0)
```

Fitting Active cases with polynomials

We fit the active cases with polynomials (with p=3,4,5).

We then calculate the evolution (based on the fitted curve) for more 14 days.

```
def plot_polinomial_fit_data(d_df):
    d_df = d_df.sort_values(by=['Date'], ascending=True)
    d_df['x'] = np.arange(len(d_df)) + 1
    d_df['y'] = d_df['Active']

    x = d_df['x']
    y = d_df['y']

    p3 =  np.poly1d(np.polyfit(x, y, 3))
    p4 =  np.poly1d(np.polyfit(x, y, 4))
    p5 =  np.poly1d(np.polyfit(x, y, 5))


    xp = range(20,d_df.shape[0] + 14)
    yp3 = p3(xp)
    yp4 = p4(xp)
    yp5 = p5(xp)

    p_df = pd.DataFrame()
    p_df['x'] = xp
    p_df['y3'] = np.round(yp3,0)
    p_df['y4'] = np.round(yp4,0)
    p_df['y5'] = np.round(yp5,0)


    traceA = go.Scatter(
        x=d_df['x'], y=d_df['y'],
        name="Active",
        marker=dict(color="Red"),
        mode = "markers+lines",
        text=d_df['Active'],
    )

    traceP3 = go.Scatter(
        x=p_df['x'], y=p_df['y3'],
        name="p = 3",
        marker=dict(color="blue"),
        mode = "lines",
```

```
        text=p_df['y3'],
    )
    traceP4 = go.Scatter(
        x=p_df['x'], y=p_df['y4'],
        name="p = 4",
        marker=dict(color="lightblue"),
        mode = "lines",
        text=p_df['y4'],
    )
    traceP5 = go.Scatter(
        x=p_df['x'], y=p_df['y5'],
        name="p = 5",
        marker=dict(color="darkblue"),
        mode = "lines",
        text=p_df['y5'],
    )


    data = [traceA, traceP3]

    layout = dict(title = 'Active cases and polynomial (p=3) curve projection (for
          xaxis = dict(title = 'Day since first case', showticklabels=True),
          yaxis = dict(title = 'Number of active cases'),plot_bgcolor='rgb(275, 27
          hovermode = 'closest'
         )
    fig = dict(data=data, layout=layout)
    iplot(fig, filename='covid-polinomial-projection')
plot_polinomial_fit_data(d_df)
```

```
test1 = test1.fillna(0)
test2 = test1.copy()
test2['cases_per_tests'] = np.round(test2['perday_tests'] / test2['perday_positive
data_daily_df = test2.replace([np.inf, -np.inf], np.nan).reset_index()
d_df = data_daily_df.copy()
```

```

from sklearn.linear_model import LinearRegression, BayesianRidge
from sklearn.ensemble import RandomForestRegressor

x = d_df[['index']]
y = d_df['cases_per_tests']

# Linear Regression
model_LR = LinearRegression()
model_LR.fit(x,y)
y1_fit = model_LR.predict(x)

# Bayesian Ridge Regression
model_BR = BayesianRidge()
model_BR.set_params(alpha_init=1.0, lambda_init=0.01)
model_BR.fit(x, y)
y2_fit = model_BR.predict(x)

# Random Forest Regression
model_RF = RandomForestRegressor(max_depth = 5, n_estimators=10)
model_RF.fit(x,y)
y3_fit = model_RF.predict(x)
```

```
traceCPTR = go.Scatter(
    x = d_df['day'],y = d_df['cases_per_tests'],
    name='Positives tests %',
    marker=dict(color='Magenta'),
    mode = "markers",
    text = d_df['cases_per_tests']
)

traceLReg = go.Scatter(
    x = d_df['day'],y = y1_fit,
    name='Linear Regression',
    marker=dict(color='Red'),
    mode = "lines",
    text = d_df['cases_per_tests']
)


traceBRReg = go.Scatter(
    x = d_df['day'],y = y2_fit,
    name='Bayesian Ridge Regression',
    marker=dict(color='Blue'),
    mode = "lines",
    text = d_df['cases_per_tests']
)

traceRFReg = go.Scatter(
    x = d_df['day'],y = y3_fit,
    name='RandomForest Regression',
    marker=dict(color='Green'),
    mode = "lines",
    text = d_df['cases_per_tests']
)

data = [traceCPTR, traceLReg, traceBRReg, traceRFReg]
layout = dict(title = 'Percent of positive tests / day (values and regression line
          xaxis = dict(title = 'Date', showticklabels=True),
          yaxis = dict(title = 'Percent of positive tests / day'),plot_bgcolor='rg
          hovermode = 'closest'
         )
fig = dict(data=data, layout=layout)
iplot(fig, filename='cases-covid19')


```

Forecasting with Prophet (Baseline)

Perfoming two week's ahead forecast with Prophet, with prediction intervals.

**Forecasting Confirmed cases with Prophet (Baseline) **

```
cnf = date_wise_data.copy()
Confirmed = cnf[['Date','Confirmed']]
Confirmed = df.groupby('Date').sum()['Confirmed'].reset_index()
Confirmed.columns = ['ds','y']
```

```
Confirmed['ds'] = pd.to_datetime(Confirmed['ds'])
dth = date_wise_data.copy()
deaths = dth[['Date','Deaths']]
deaths = df.groupby('Date').sum()['Deaths'].reset_index()
deaths.columns = ['ds','y']
deaths['ds'] = pd.to_datetime(deaths['ds'])
```

```
from fbprophet import Prophet
from fbprophet.plot import plot_plotly
```

```
m= Prophet(interval_width=0.99)
m.fit(Confirmed)
future = m.make_future_dataframe(periods=14)
future_confirmed = future.copy() # for non-baseline predictions later on
forecast = m.predict(future)
forecast = forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
```

```
fig = plot_plotly(m, forecast)
fig.update_layout(title_text = 'Confirmed cases Prediction using prophet')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
py.iplot(fig)
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=date_wise_data['Confirmed'],
                    mode='lines+markers',marker_color='blue',name='Actual'))
fig.add_trace(go.Scatter(x=forecast['ds'], y=forecast['yhat_upper'],
                    mode='lines',marker_color='Orange',name='Predicted'))
fig.update_layout(title_text = 'Confirmed cases Predicted vs Actual using prophet'
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
fig.show()
```

**Forecasting Deaths with Prophet (Baseline) **

```
md= Prophet(interval_width=0.99)
md.fit(deaths)
futured = md.make_future_dataframe(periods=14)
future_confirmed = futured.copy()
forecastd = md.predict(futured)
forecastd = forecastd[['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
```

```
fig = plot_plotly(md, forecastd)
fig.update_layout(title_text = 'Deaths Prediction using prophet')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
py.iplot(fig)
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=date_wise_data['Date'], y=date_wise_data['Deaths'],
                    mode='lines+markers',marker_color='blue',name='Actual'))
fig.add_trace(go.Scatter(x=forecastd['ds'], y=forecastd['yhat_upper'],
                    mode='lines',marker_color='red',name='Predicted'))
fig.update_layout(title_text = 'Deaths Predicted vs Actual using prophet')
fig.update_layout(plot_bgcolor='rgb(275, 270, 273)',width=600, height=600)
fig.show()
```

Together We can do this.

Note: This work is highly inspired from few other kaggle kernels , github sources and other data science resources. Any traces of replications, which may appear , is purely co-incidental. Due respect & credit to all my fellow kagglers.